

Milestone Compliance Report 1

Heterogeneous CPU-GPU Implementation of Collision Detection for Gaming

Abstract:

This MCR presents a heterogeneous CPU-GPU implementation of collision detection for gaming applications, focusing on fluid simulation and particle physics. We describe our approach to workload distribution between CPU and GPU, initially achieving an average CPU frame time of 6.00ms (166.7 FPS) with optimized data transfers. Key improvements include replacing parallel bitonic sorting with counting sort for spatial subdivision, implementing array copyback kernels for better memory locality, and adding support for multiple collision types. The implementation features dynamic particle activation, multiple fluid types, and efficient GPU-CPU communication through Unity's AsyncGPUReadback. We demonstrate the practical application of these optimizations in a game environment, complete with level progression, scoring systems, and cross-platform UI controls. Performance analysis shows up to 200% improvement in specific scenarios after our optimizations, with the system successfully handling 10,000 particles at 1000 FPS in the best case.

CPU-GPU Workload Split:

For this implementation we wanted to focus on ensuring that the CPU could obtain adequate performance in the 3 most computationally heavy kernels, allowing the GPU to continue to perform the other 4 kernels. We also needed to implement a method of sorting the uniform subdivision cells by their population to allow the CPU to work on only the highest population cells.

The first implementation of our CPU and GPU workload split involved moving the density, pressure, and viscosity kernels to the CPU, as well as the collisions between large objects. In order to move the three kernels to the CPU the kernels had to be rewritten to work with the Unity Jobs system. The Unity Jobs system allows for efficient scheduling and synchronization of CPU threads within the Unity engine, allowing for parallel execution of threads over every CPU core [1]. With the CPU kernels created, the spatial subdivision sorting algorithm was then modified to also calculate the population of each subdivision cell and sort them in descending order. The current pipeline for this CPU-GPU implementation is shown in figure 1.

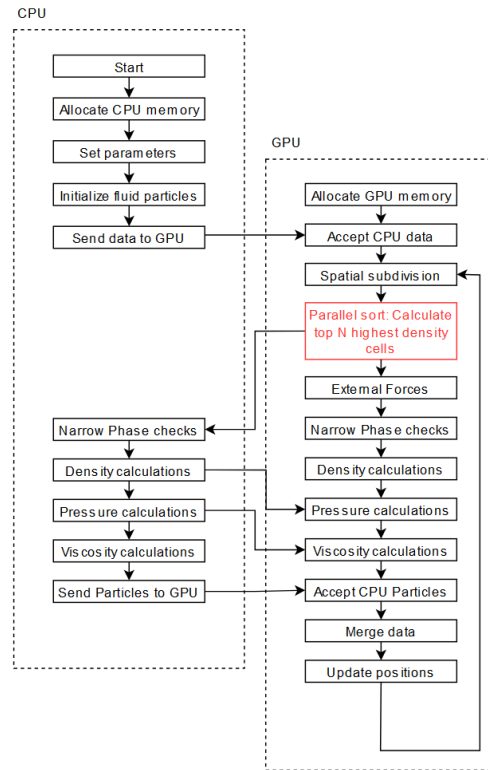


Figure 1: CPU-GPU Pipeline V1

The GPU performs the spatial subdivision sorting and calculates the highest population cells, then the data is transferred to the CPU which computes the density, pressure and viscosity of the highest population cells and sends the results to the GPU to be merged with the rest of the particle data. A performance analysis of the CPU-GPU implementation was then performed, showing the CPU-GPU implementation had an average CPU frame time of 6.00ms or 166.7fps. About 4ms was spent on CPU-GPU data transfers, 0.25ms was spent on CPU computation, 0.10ms was spent on GPU computation (waiting for GPU). The remaining frame time was spent on management tasks for the game environment.

Overview	Total	Self	Calls	GC Alloc	Time ms
► Gfx.GetComputeBuffer	76.8%	0.1%	18	0 B	4.19
► JobHandle.Complete	4.7%	0.3%	9	0 B	0.25
Compute.Dispatch	1.8%	1.8%	666	0 B	0.10

Figure 2: CPU-GPU single frame times, First entry shows data transfer time and calls, second entry shows CPU computation time, third shows GPU computation time.

The result for the first implementation shows a good path to performance optimization, as we move more kernels to the CPU, the data transfers time should decrease and CPU computation time increase. The goal for the next MCR is to move the spatial subdivision sorting to the CPU, which should remove roughly 20 - 30% of the data transfers, as well as modifying the existing CPU-GPU data transfers to only pass the specific type of data needed for each kernel.

Algorithmic & Performance Optimizations:

Our first implementation of the fluid simulation used parallel bitonic sorting for the broad-phase spatial subdivision algorithm, and only sorted the indices of arrays storing the particle data, rather than rearranging the memory locations. The former algorithm was simply not the most efficient sorting algorithm to implement, while the latter led to poor performance due to both poor spatial locality when particle data was accessed for use by the physics kernels.

The initial sorting algorithm, bitonic sort, sorts by comparing and swapping pairs of (bitonic) sequences, merging them at each stage to produce a sorted dataset. It is a relatively inefficient algorithm in parallel contexts due to its poor performance at scale (with $O(n \log^2(n))$ complexity) as well as large overhead with many parallel stages (due to the inherent data dependencies that stem from needing to merge a pair of sorted sequences repetitively). These limitations gimp the amount of performance that can be achieved when running the algorithm for many elements across many parallel stages, drastically limiting performance both as n becomes large and as the degree of parallelism increases.

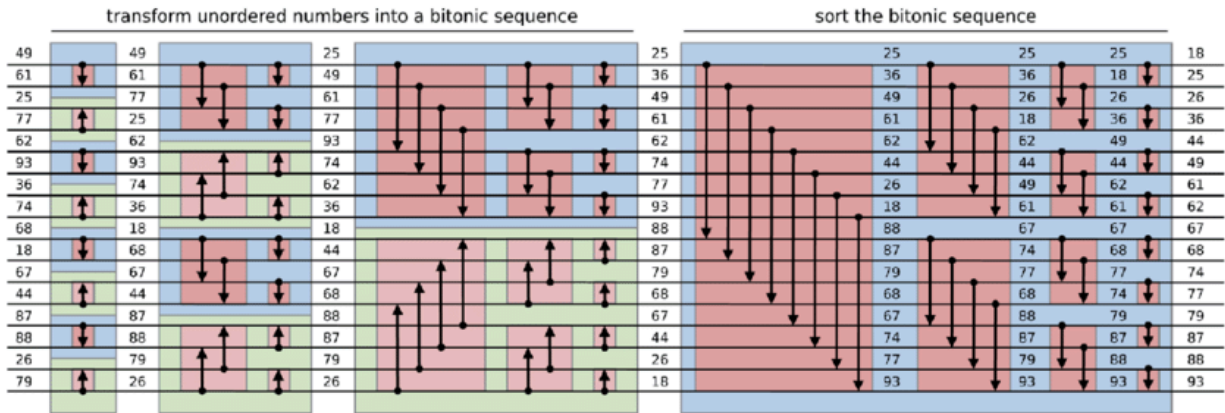


Figure 3: Visual depiction of bitonic sort [2].

Conversely, counting sort, sorts by computing the position or rank of each element through a count array (in this case, the count of particles in each grid cell), with final element positions determined by summation. The algorithm has very few data dependencies, a better time complexity ($O(n+k)$ (where k is the array size)) and scales very well in parallel contexts; primarily due to low memory overhead as well as the usage of GPU prefix sums when determining cumulative counts for array index position (which can utilize redundant counts to achieve faster overall sorting). Implementing counting sort led to a noticeable overall performance increase of around 10-20% (dependent on system).

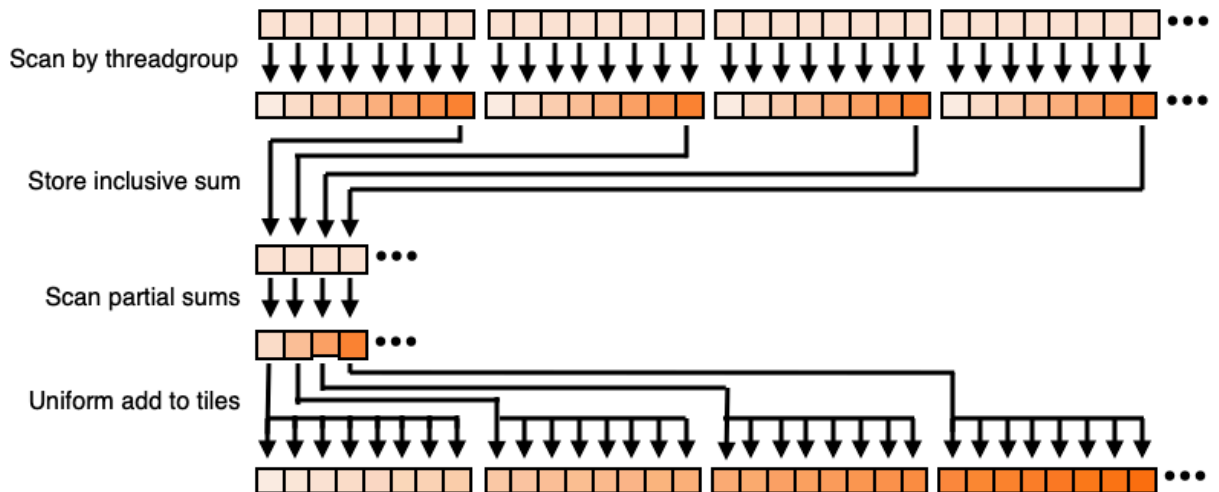
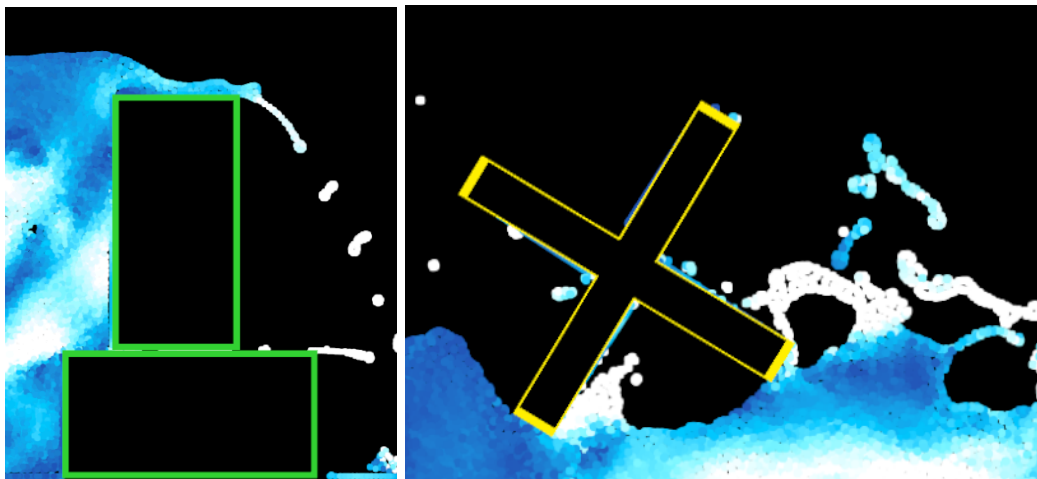


Figure 4: Visual depiction of counting sort [3].

Lastly, we implemented array copyback kernels to actually sort array memory contents instead of just their indices. The main motivation being to improve the spatial locality of all physics kernels. This simple change resulted in monstrous improvements in performance — up to almost 200% (580 FPS to 1000 FPS on a 10k particle simulation). We predict that kernel performance on the CPU will benefit the most from this change due to the nature of branch prediction algorithms and the advantages they may gain from good spatial and temporal locality.

Other Game Features and Improvements:

Support for box colliders: We implemented oriented box colliders in order to add different possible shapes to our simulation. The concept was mentioned in passing in MCR 3 from the fall semester. We are able to combine multiple boxes to simulate more complex objects such as the pinwheel in our sandbox demo level.



Figures 5.1 and 5.2: On the left are two separate box colliders and on the right is a pinwheel created by connecting two box colliders at a central pivot point.

Support for circle colliders: As mentioned in previous papers, our fluid particles are represented by smoothed circular colliders and their calculations are run in parallel using a uniform spatial hashing grid for the broadphase and bounding sphere checks for the narrow phase. As mentioned in those papers, one of the weaknesses of this approach is that the uniform grid must accommodate the largest colliders being simulated which causes larger colliders to adversely affect the number of neighbour checks each particle must run, slowing the overall simulation. We have added a way for a small quantity of larger circle colliders to run on the CPU separately from the kernels responsible for the rest of the fluids, this isolates them from the broad phase uniform grid step of computation and thus for small quantities of colliders, improves overall performance.

Support for activating and deactivating particles: One of the necessary features that our simulation needed to have in order to turn it into a game was the ability to dynamically change the number of particles being simulated at runtime. This would be trivially easy in a regular CPU-bound program because you can allocate and deallocate memory at will; however, in our implementation which combines the CPU and GPU we needed to allocate memory ahead of time in order to maintain memory coherence and locality. To achieve our goal, we created a struct which contained all the properties of a particle and included a flag which we would check at the start of each kernel, if the flag indicated that the particle was disabled then all the calculations of that kernel were able to be skipped. The entire array of struct data was also passed to the visual shader so that it could use dynamic branching to hide disabled particles. In order to improve memory locality instead of creating other flags we also used this same flag to check the particle type, this enabled us to support multiple different fluid properties.

Support for Multiple Fluid Types: By altering the physical and visual properties of each particle we are able to simulate different types of fluids dynamically at runtime. But in order to maintain memory coherence and spatial locality we cannot store the properties of each particle within the particle struct itself. That would lead to less cache hits as the particle structure would be spaced further apart in memory and increased memory usage as data is duplicated. In order to prevent this issue, we take a data oriented approach and first create a general fluid class which each fluid type implements. Then the compute shader responsible for physics calculations and the visual shader responsible for drawing each particle to the screen are sent a table of all the properties needed for each possible particle fluid type in each scene at start-time. The compute shader receives an array of structs, each struct in the array containing the physical properties of a single type of fluid. The visual shader receives a similar array of structs containing visual properties for each fluid type but it also needs to do some preprocessing of this data. Each particle contained a one dimensional gradient texture that was 64px by 1px in resolution which was used for visual effects such as changing the particle's color depending on its speed, this data from each particle was stitched together in order to form a single texture which would be easier for the visual shader to work with, so for a scene with 7 fluid types, a 64x7 resolution 2D texture would be created. For both physical and visual shaders, each thread referenced the data it needed from the tables by checking the single integer flag that was within

each particle struct. There is one thread per particle and each wavefront contains a set of 32 to 64 threads that loads the entire table that it needs into its memory to make references faster.

Fluid Density Detector: To create an actual game there needs to be asynchronous communication between the CPU and the GPU. This is so that the CPU can check on the state of the particles in a scene every once in a while and then update the UI or game state as required without blocking the GPU and creating discernable slowdowns. Our fluid sensor which can be used as a trigger to start a level completion sequence takes advantage of the AsyncGPUReadback function that is provided by the unity engine in order to copy back a buffer of particle positions from the GPU without stalling at the expense of some latency. The fluid sensor then computes the density of particles surrounding a specific point in the game world coordinate system. This could be the position of a target or of a fire or the center of some container. Once the density around that area has surpassed a threshold that the level creator has set, the fluid sensor can then send a signal to the level management system to trigger that the level has been completed or to change the game state in some other manner.

Audio Playback System: We used Unity Engine built in system to create a dual channel audio mixer setup so that our game could have both background music and in game sound effects playing over each other and also independently controlled via the user's settings menu.

Level Scoring System: The scoring system is currently based on time to finish a level. Simply finishing a level will award one star, while completing a level under a certain time threshold will award two or potentially three stars. This is potentially subject to change as we add more levels. However, alternative scoring systems will likely be specific to the particular level, so we can't speculate further at this time. The level completion information as well as user preferences are saved to the user's disk into the user's platform registry without encryption, this data is stored differently based on the operating system that the application runs on.

Main Menu and Level Selection Pages: Currently, the main menu has four options, Play, Sandbox Mode, Settings, and Quit. The play button opens a list of levels ("Level Selection") and is intended to be how the player enters the different experiences that our game offers. The level selection screen also showcases the player's past progress and performance in the levels they have completed based on the star scoring system.

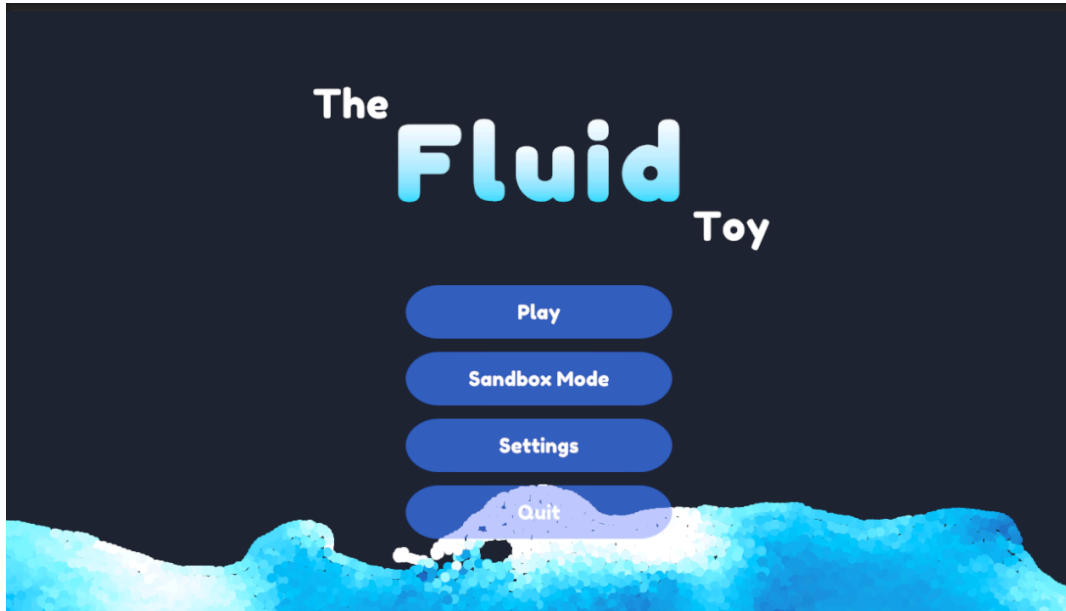


Figure 6: The Main Menu Screen

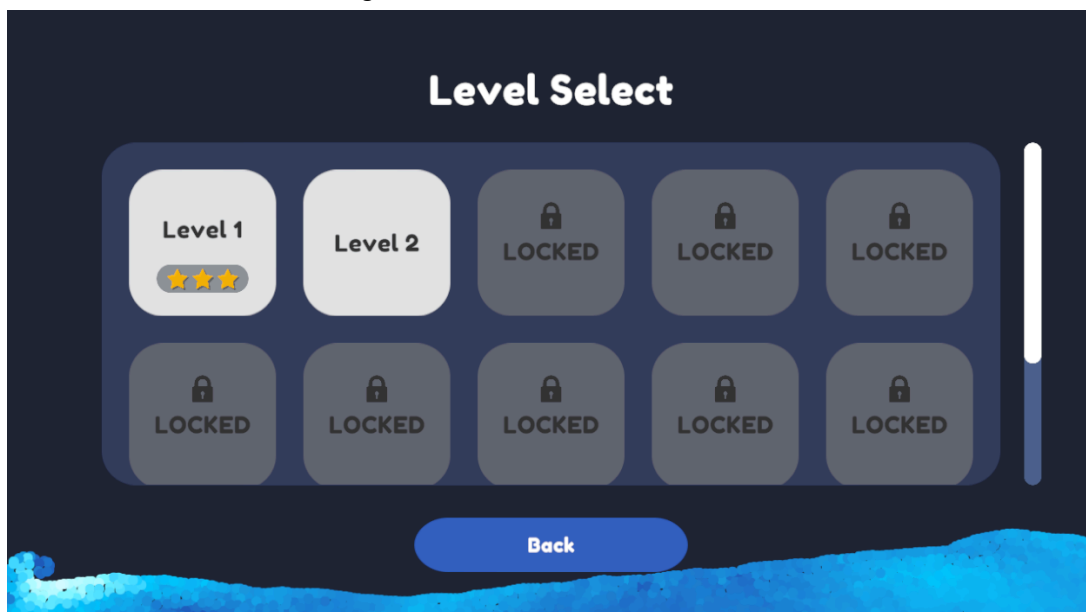


Figure 7: The Level Selection Screen

Pause Menu: The pause menu is important so that the user can temporarily halt the game state whenever they want to and to resume it at will. It allows for a place for the user to adjust their settings such as the game volume and to give up on a level and return to the main menu.

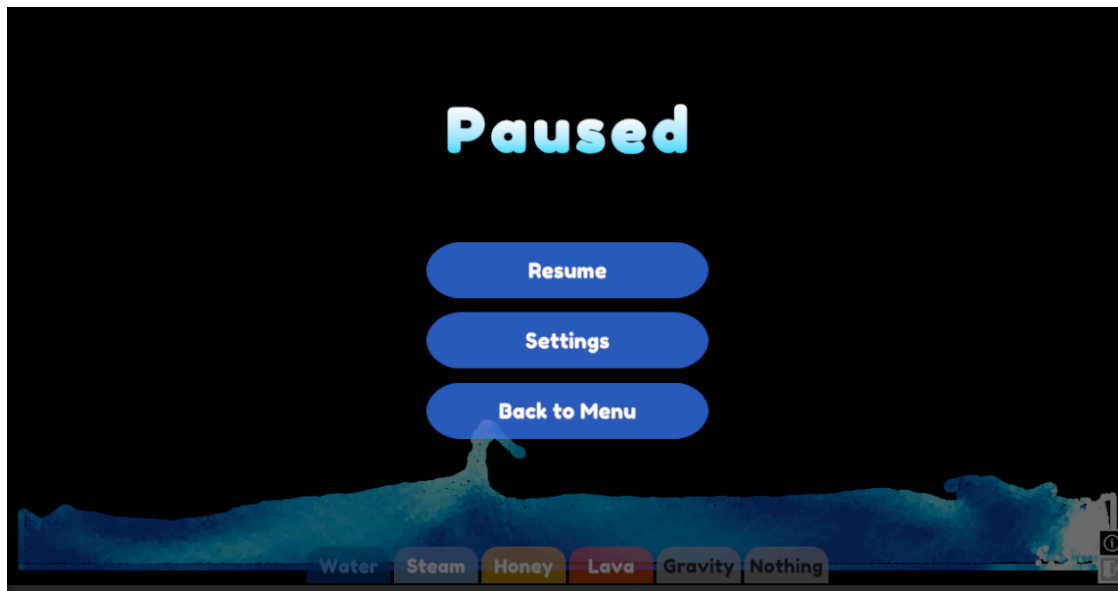


Figure 8: Image of the Pause Menu

Settings Page: This page allows the user to set the window size and resolution as well as if the window should be fullscreen or not. This is important for supporting different types of monitors. The user can also adjust the refresh rate of the simulation which affects how smooth the particles appear to move. We also added sliders which adjust the background music volume and the sound effect volume separately. There are also buttons which allow developers to unlock or lock all the levels for faster testing and development.

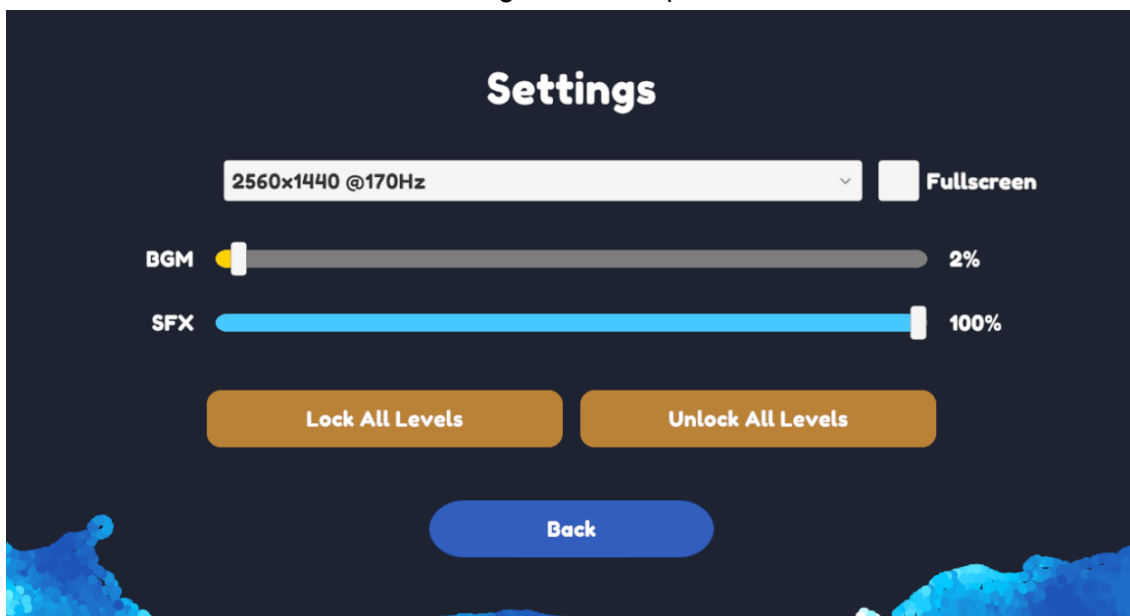


Figure 9: Image of the Settings Menu

Started Working on Visual Shaders: We send an array of particle structs which each contain a flag indicating the type of fluid that each particle should be as well as a table which contains the visual properties of each fluid type to the vertex and fragment shaders for them to be drawn to the screen. This takes advantage of the dynamic branching capabilities of modern GPUs, instead of having multiple different shaders for the different fluid types there only needs to be one shader which changes the drawing behavior depending on the type of each particle which gets its own thread. We also use two draw passes right now with different alpha blending modes in order to achieve a glowing effect around certain particles but this is currently under development.

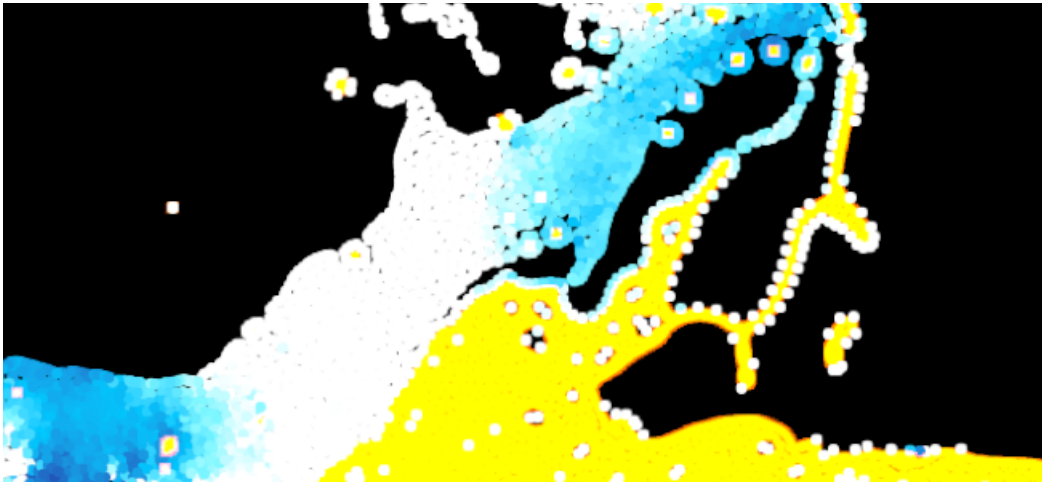


Figure 10: Water and Lava particles Clashing against each other (Temperature simulation not yet implemented)

User Interface Controls: We want to create a simple control scheme that works on desktop computers, laptops, tablets, and mobile devices. This will take advantage of Unity's cross compilation features as well as demonstrate the possibilities of utilizing the GPUs in lower end hardware. One consideration that we need to make is support for both mouse and touchscreen inputs. Our application will allow keyboard shortcuts but also any keyboard shortcut will need a graphical UI element so that users on mobile devices can access the same functionality through their touchscreens. For now, we have a tray at the bottom of the screen and a secondary tray on the bottom right of the screen. The bottom most tray will have buttons which allow the user to change their current brush (if the level needs that functionality) and the right tray will contain buttons that allow the user to pause, access level settings, help buttons, hint buttons, etc.

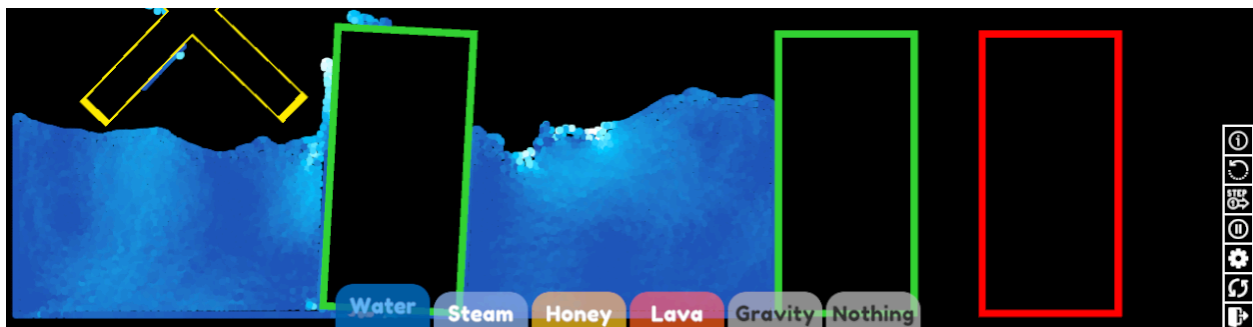


Figure 11: User Interface controls for manipulating/creating fluids, and controlling the scene.

Game Content:

Level 1: This is the first level of our game. Each level will slowly increase in complexity allowing us to gently guide the user through the different functions of our fluid simulation engine. For this level the goal is to use the gravity brush to fill a cup with fluid. The fluid is themed to look like beer and the level has a tavern aesthetic. We will agitate the beer to make the level more chaotic and fun and the user will be timed on how fast they can fill the cup with beer. Once the cup is filled a fluid detector will trigger the level completion animations which will indicate that the level is done and it will display how well the player did. The high score of the player will then be saved to the disk and displayed on the level select screen.

In terms of implementation details, the cup's movements are simulated on the CPU. Its position, size and shape is then sent to the GPU where all the fluid particles are also simulated. Collisions between the fluid and the cup are calculated on the GPU and the fluid particle movements are updated in parallel on the GPU. Then the GPU sends back the positions of the fluid particles to the CPU which checks if the cup has been filled with fluid yet to end the level. We've play tested it with some friends and family who found it fun and engaging.

We plan to create at least 10 more levels and to work on further improving and optimizing the fluid simulation capabilities.

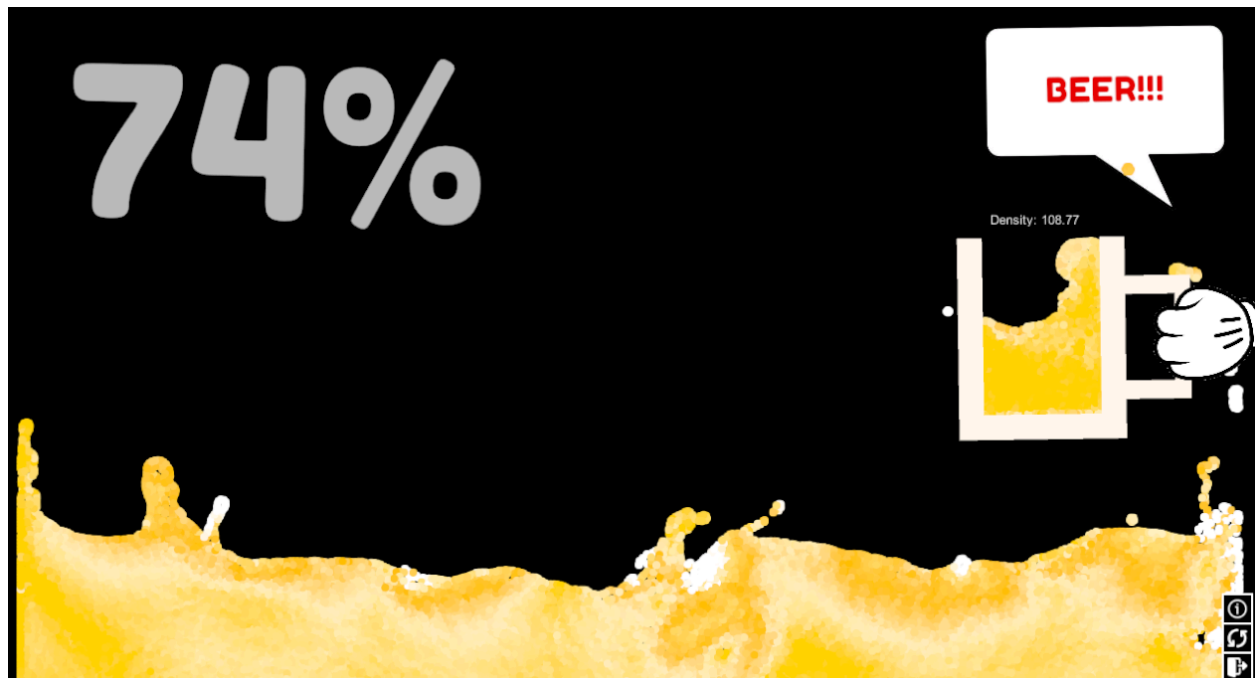


Figure 12: Demo of Level 1, the beer is at the bottom and the goal is to use the mouse to fill the cup with beer.

Sandbox Mode:

This is the sandbox level. This level was designed for two purposes, both as a functional sandbox where the player can use and experiment with all of the fluids and objects that we have developed for the game, and as a benchmark level that we use to evaluate the performance of the fluid simulation. This level will be updated to include more features as we develop the game, allowing us to test all features in a single environment, streamlining the testing process.

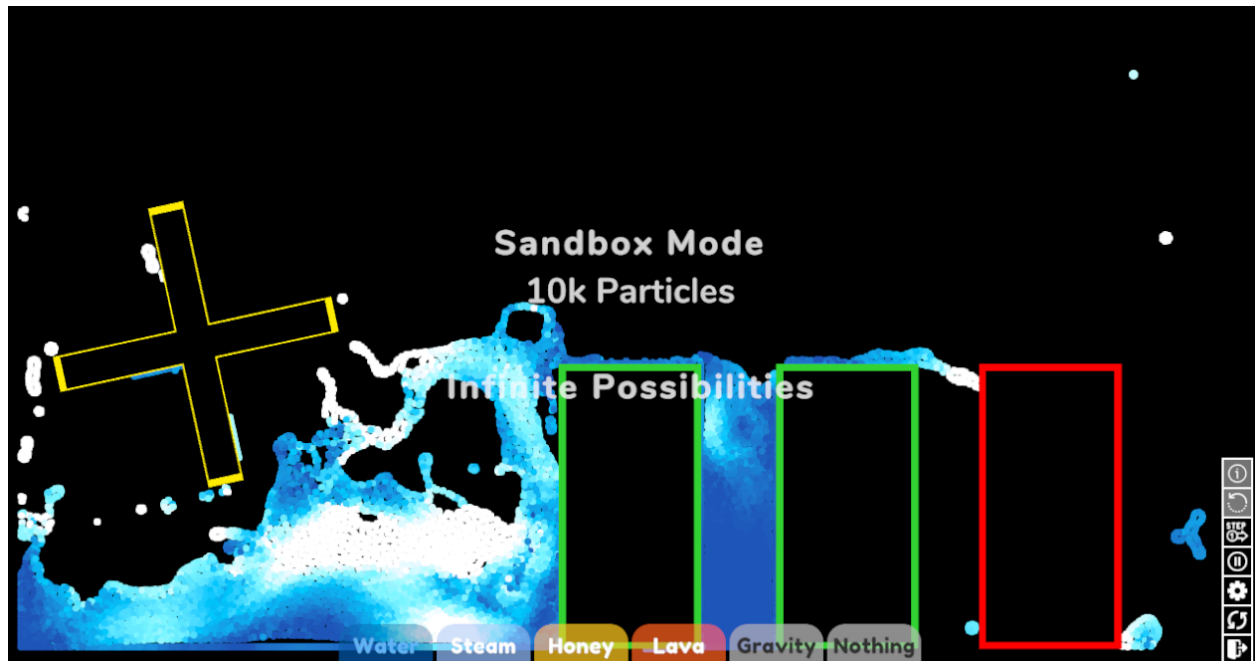


Figure 13: The Sandbox level used for both benchmarking and for player experimentation.

Conclusion:

Our heterogeneous CPU-GPU implementation demonstrates significant improvements in both performance and functionality. The transition from bitonic sort to counting sort, coupled with optimized array copyback kernels, yielded substantial performance gains, achieving up to 200% improvement in frame rates. Our implementation successfully handles complex fluid simulations with 10,000 particles at 1000 FPS, while supporting multiple fluid types and collision detection mechanisms.

The workload distribution between CPU and GPU proved effective, with initial implementations achieving 166.7 FPS and showing clear paths for further optimization through reduced data transfers and improved kernel distribution. The addition of features such as dynamic particle activation, multiple fluid types, and cross-platform UI controls has transformed the technical implementation into a functional game environment with robust level progression and scoring systems.

Future work will focus on moving spatial subdivision sorting to the CPU to reduce data transfer overhead by 20-30%, optimizing CPU-GPU data transfers for specific kernel requirements, and implementing temperature simulation for fluid interactions. Additional game levels will be developed to fully showcase the capabilities of our fluid simulation system while maintaining performance across various hardware configurations.

References:

- [1] U. Technologies, "Job system overview," Unity, <https://docs.unity3d.com/Manual/job-system-overview.html> (accessed Jan. 21, 2025).
- [2] "Sorting network for the bitonic sort," *ResearchGate*. Available: <https://www.researchgate.net/publication/342402983/figure/fig1/AS:906061875798017@1593033393665/Sorting-network-for-the-bitonic-sort-Each-horizontal-line-represents-an-input-and-each.png>. [Accessed: Jan. 22, 2025].
- [3] "Parallel Sorting on GPU," *Medium*. Available: https://miro.medium.com/v2/resize:fit:1400/1*uiuxN9SyZJPcIT22uTUcMw.png. [Accessed: Jan. 22, 2025].